CYBER SECURITY

Malicious Software

Er. Prakash Chandra Adhikari

5/3/2026

1 VIRUSES AND RELATED THREATS

1.1 MALICIOUS PROGRAMS (or Taxonomy of Malicious Programs)

- Figure 1 provides an overall taxonomy of software threats or malicious programs.
- Malicious software can be divided into two categories: those that need a host program, and those that are independent.
- The former are essentially fragments of programs that cannot exist independently of some actual application program, utility, or system program. Viruses, logic bombs, and backdoors are examples.
- The latter are self contained programs that can be scheduled and run by the operating system.Worms and zombie programs are examples.

Name	Description
Virus	Attaches itself to a program and propagates copies of itself to other programs
Worm	Program that propagates copies of itself to other computers
Logic bomb	Triggers action when condition occurs
Trojan horse	Program that contains unexpected additional functionality
Backdoor (trapdoor)	Program modification that allows unauthorized access to functionality
Exploits	Code specific to a single vulnerability or set of vulnerabilities
Downloader's	Program that installs other items on a machine that is under attack. Usually, a Downloader is sent in an e-mail.
Auto-rooter	Malicious hacker tools used to break into new machines remotely
Kit (virus generator)	Set of tools for generating new viruses automatically

Spammer programs	Used to send large volumes of unwanted e-mail
Flooders	Used to attack networked computer systems with a large volume of traffic to carry out a denial of service (DoS) attack
Key loggers	Captures keystrokes on a compromised system
Root kit	Set of hacker tools used after attacker has broken into a computer system and gained root-level access
Zombie	Program activated on an infected machine that is activated to launch attacks on other machines

Table 1 Terminology of Malicious Programs

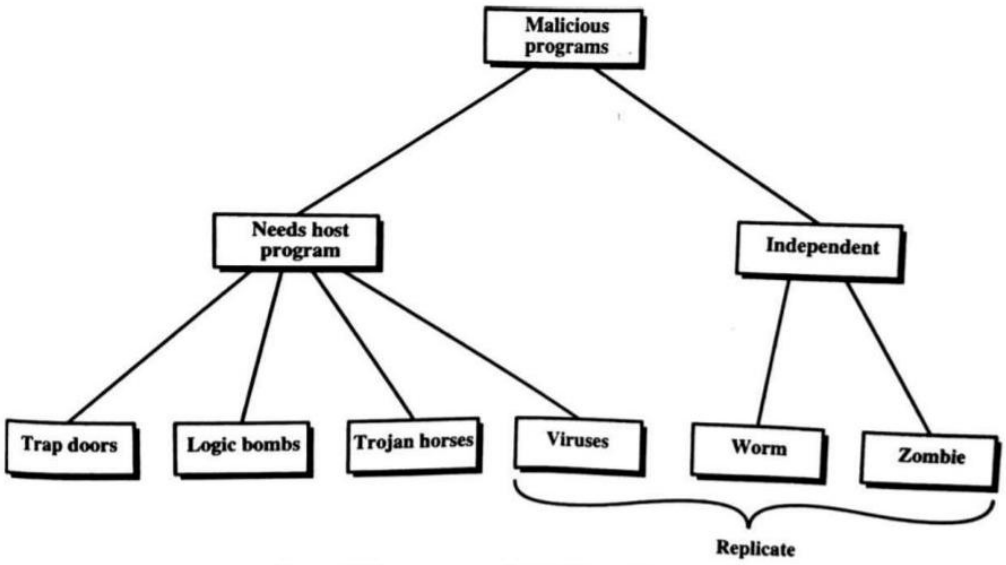

Figure1 Taxonomy of Malicious Programs

- We can also differentiate between those software threats that do not replicate and those that do.
- The former are programs or fragments of programs that are activated by a trigger. Examples are logic bombs, backdoors, and zombie programs.

- The latter consist of either a program fragment or an independent program that, when executed, may produce one or more copies of itself to be activated later on the same system or some other system. Viruses and worms are examples.

1.1.1 Trapdoor or back door

- A trap door is a secret entry point into a program that allows someone that is aware of the trap door to gain access without going through the usual security access procedures.
- Also, we can also say that it is a method of bypassing normal authentication methods.
- It is also known as a back door.
- Trap doors have used legally for many years by programmers to debug and test programs.
- Moreover, Trap doors become threats when they used by dishonest programmers to gain unauthorized access.
- It is difficult to implement operating system controls for trap doors.
- Security measures must focus on the program development and software update activities.

1.1.2 Logic Bomb

- One of the oldest types of program threat, predating viruses and worms, is the logic bomb.
- The logic bomb is code embedded in some legitimate program that is set to "explode" when certain conditions are met.
- Examples of conditions that can be used as triggers for a logic bomb are the presence or absence of certain files, a particular day of the week or date, or a particular user running the application. Once triggered, a bomb may alter or delete data or entire files, cause a machine halt, or do some other damage.

1.1.3 Trojan Horses

- A Trojan horse is a useful, or apparently useful, program or command procedure containing hidden code that, when invoked, performs some unwanted or harmful function.
- Trojan horse programs can be used to accomplish functions indirectly that an unauthorized user could not accomplish directly. For example, to gain access to the

files of another user on a shared system, a user could create a Trojan horse program that, when executed, changes the invoking user's file permissions so that the files are readable by any user. The author could then induce users to run the program by placing it in a common directory and naming it such that it appears to be a useful utility program or application. An example is a program that ostensibly produces a listing of the user's files in a desirable format. After another user has run the program, the author of the program can then access the information in the user's files. An example of a Trojan horse program that would be difficult to detect is a compiler that has been modified to insert additional code into certain programs as they are compiled, such as a system login program [THOM84]. The code creates a backdoor in the login program that permits the author to log on to the system using a special password. This Trojan horse can never be discovered by reading the source code of the login program.

- Another common motivation for the Trojan horse is data destruction. The program appears to be performing a useful function (e.g., a calculator program), but it may also be quietly deleting the user's files. For example, a CBS executive was victimized by a Trojan horse that destroyed all information contained in his computer's memory [TIME90].
- The Trojan horse was implanted in a graphics routine offered on an electronic bulletin board system.

1.1.4 Zombie

- A zombie is a program that secretly takes over another Internet-attached computer and then uses that computer to launch attacks that are difficult to trace to the zombie's creator.
- Zombies are used in denial of- service attacks, typically against targeted Web sites.
- The zombie is planted on hundreds of computers belonging to unsuspecting third parties, and then used to overwhelm the target Web site by launching an overwhelming onslaught of Internet traffic.

1.2 The Nature of Viruses

- A virus is a piece of software that can "infect" other programs by modifying them; the modification includes a copy of the virus program, which can then go on to infect other programs.
- Biological viruses are tiny scraps of genetic code DNA or RNA that can take over the machinery of a living cell and trick it into making thousands of flawless replicas of the original virus. Like its biological counterpart, a computer virus carries in its instructional code the recipe for making perfect copies of itself.
- A virus can do anything that other programs do. The only difference is that it attaches itself to another program and executes secretly when the host program is run. Once a virus is executing, it can perform any function, such as erasing files and programs.
- During its lifetime, a typical virus goes through the following four phases: **JUNE - 2012[10M]**
  - **Dormant phase:** The virus is idle. The virus will eventually be activated by some event, such as a date, the presence of another program or file, or the capacity of the disk exceeding some limit. Not all viruses have this stage.
  - **Propagation phase:** The virus places an identical copy of itself into other programs or into certain system areas on the disk. Each infected program will now contain a clone of the virus, which will itself enter a propagation phase.
  - **Triggering phase:** The virus is activated to perform the function for which it was intended. As with the dormant phase, the triggering phase can be caused by a variety of system events, including a count of the number of times that this copy of the virus has made copies of itself.
  - **Execution phase:** The function is performed. The function may be harmless, such as a message on the screen, or damaging, such as the destruction of programs and data files.
- Most viruses carry out their work in a manner that is specific to a particular operating system and, in some cases, specific to a particular hardware platform. Thus, they are designed to take advantage of the details and weaknesses of particular systems.

1.2.1 Virus structure

- A virus can be prep ended or post pended to an executable program, or it can be embedded in some other fashion. The key to its operation is that the infected program, when invoked, will first execute the virus code and then execute the original code of the program.
- A very general depiction of virus structure is shown in Figure 2 (based on [COHE94]. In this case, the virus code, V, is prep ended to infected programs, and it is assumed that the entry point to the program, when invoked, is the first line of the program.

```
        program V :=

{goto main;
        1234567;

            subroutine infect-executable :=
                {loop:
                file := get-random-executable-file;
                if (first-line-of-file = 1234567)
                        then goto loop
                        else prepend V to file; }

            subroutine do-damage :=
                {whatever damage is to be done}

            subroutine trigger-pulled :=
                {return true if some condition holds}

main:       main-program :=
                {infect-executable;
                if trigger-pulled then do-damage;
                goto next;}

next:

}
```

Figure2 A Simple Virus

- A virus such as the one just described is easily detected because an infected version of a program is longer than the corresponding uninfected one. A way to thwart such a simple means of detecting a virus is to compress the executable file so that both the infected and uninfected versions are of identical length.
- Figure 3 [COHE94] shows in general terms the logic required.
- The key lines in this virus are numbered, and Figure 4 [COHE94] illustrates the operation. We assume that program P1 is infected with the virus CV. When this program is invoked, control passes to its virus, which performs the following steps:
  1. For each uninfected file P2 that is found, the virus first compresses that file to produce P'2, which is shorter than the original program by the size of the virus.

2. A copy of the virus is prep ended to the compressed program.
3. The compressed version of the original infected program, P'1, is uncompressed.
4. The uncompressed original program is executed.

```
        program CV :=

{goto main;
        01234567;

          subroutine infect-executable :=
                        {loop:
                                file := get-random-executable-file;
                        if (first-line-of-file = 01234567) then goto loop;
                (1)     compress file;
                (2)     prepend CV to file;
                }

main:  main-program :=
                        {if ask-permission then infect-executable;
                (3)     uncompress rest-of-file;
                (4)     run uncompressed file;}
                }
```

Figure 3 Logic for a Compression Virus

1.2.2 Initial Infection

Once a virus has gained entry to a system by infecting a single program, it is in a position to infect some or all other executable files on that system when the infected program executes. Thus, viral infection can be completely prevented by preventing the virus from gaining entry in the first place. Unfortunately, prevention is extraordinarily difficult because a virus can be part of any program outside a system. Thus, unless one is content to take an absolutely bare piece of iron and write all one's own system and application programs, one is vulnerable.

1.2.3 Types of Viruses

- There has been a continuous arms race between virus writers and writers of antivirus software since viruses first appeared. As effective countermeasures have been developed for existing types of viruses, new types have been developed. [STEP93] suggests the following categories as being among the most significant types of viruses:
  1. **Parasitic virus:** The traditional and still most common form of virus. A parasitic virus attaches itself to executable files and replicates, when the infected program is executed, by finding other executable files to infect.
  2. **Memory-resident virus:** Lodges in main memory as part of a resident system program. From that point on, the virus infects every program that executes.
  3. **Boot sector virus:** Infects a master boot record or boot record and spreads when a system is booted from the disk containing the virus.
  4. **Stealth virus:** A form of virus explicitly designed to hide itself from detection by antivirus software.
  5. **Polymorphic virus:** A virus that mutates with every infection, making detection by the "signature" of the virus impossible.
  6. **Metamorphic virus:** As with a polymorphic virus, a metamorphic virus mutates with every infection. The difference is that a metamorphic virus rewrites itself completely at each iteration, increasing the difficulty of detection. Metamorphic viruses my change their behavior as well as their appearance.
- One example of a **stealth virus**: a virus that uses compression so that the infected program is exactly the same length as an uninfected version. Far more sophisticated techniques are possible.

- A **polymorphic virus** creates copies during replication that are functionally equivalent but have distinctly different bit patterns. As with a stealth virus, the purpose is to defeat programs that scan for viruses. In this case, the "signature" of the virus will vary with each copy. To achieve this variation, the virus may randomly insert superfluous instructions or interchange the order of independent instructions. A more effective approach is to use encryption. The strategy of the encryption virus is followed. The portion of the virus that is responsible for generating keys and performing encryption/decryption is referred to as the mutation engine. The mutation engine itself is altered with each use.

1.2.4 Macro Viruses

- In the mid-1990s, macro viruses became by far the most prevalent type of virus. Macro viruses are particularly threatening for a number of reasons:
  1. A macro virus is platform independent. Virtually all of the macro viruses infect Microsoft Word documents. Any hardware platform and operating system that supports Word can be infected.
  2. Macro viruses infect documents, not executable portions of code. Most of the information introduced onto a computer system is in the form of a document rather than a program.
  3. Macro viruses are easily spread. A very common method is by electronic mail.
  - Macro viruses take advantage of a feature found in Word and other office applications such as Microsoft Excel, namely the macro.
  - a macro is an executable program embedded in a word processing document or other type of file. Typically, users employ macros to automate repetitive tasks and thereby save keystrokes.
  - The macro language is usually some form of the Basic programming language.
  - In Microsoft word, there are three types of auto executing macros
    - **Auto execute:** if a macro named auto Exec is in the " normal.dot" template or in a global template stored in words start-up directory, it is executed whenever Word is started.
    - **Auto macro:** an auto macro executes when a defined event occurs, such as opening or closing a document, creating a new documents or quitting Word.
    - **Command macro:** if a macro in a global macro file or a macro attached to a document has the name of an existing word command, it is executed whenever the user invokes that command (e.g. File saves).

1.2.5 E-mail Viruses

A more recent development in malicious software is the e-mail virus. The first rapidly spreading e-mail viruses, such as Melissa, made use of a Microsoft Word macro embedded in an attachment. If the recipient opens the e-mail attachment, the Word macro is activated. Then

1. The e-mail virus sends itself to everyone on the mailing list in the user's e-mail package.
2. The virus does local damage.

At the end of 1999, a more powerful version of the e-mail virus appeared. This newer version can be activated merely by opening an e-mail that contains the virus rather than opening an attachment. The virus uses the Visual Basic scripting language supported by the e-mail package.

Thus we see a new generation of malware that arrives via e-mail and uses e-mail software features to replicate itself across the Internet. The virus propagates itself as soon as activated (either by opening an e-mail attachment of by opening the e-mail) to all of the e-mail addresses known to the infected host. As a result, whereas viruses used to take months or years to propagate, they now do so in hours. This makes it very difficult for antivirus software to respond before much damage is done. Ultimately, a greater degree of security must be built into Internet utility and application software on PCs to counter the growing threat.

1.3 Worms

- A worm is a program that can replicate itself and send copies from computer to computer across network connections.
- Upon arrival, the worm may be activated to replicate and propagate again.
- In addition to propagation, the worm usually performs some unwanted function.
- An e-mail virus has some of the characteristics of a worm, because it propagates itself from system to system.
- A worm actively seeks out more machines to infect and each machine that is infected serves as an automated launching pad for attacks on other machines.
- Network worm programs use network connections to spread from system to system.

- To replicate itself, a network worm uses some sort of network vehicle. Examples include the following:

1. **Electronic mail facility:** A worm mails a copy of itself to other systems.
2. **Remote execution capability:** A worm executes a copy of itself on another system.
3. **Remote login capability:** A worm logs onto a remote system as a user and then uses commands to copy itself from one system to the other.

- The new copy of the worm program is then run on the remote system where, in addition to any functions that it performs at that system, it continues to spread in the same fashion.
- A network worm exhibits the same characteristics as a computer virus: a dormant phase, a propagation phase, a triggering phase, and an execution phase. The propagation phase generally performs the following functions:

1. Search for other systems to infect by examining host tables or similar repositories of remote system addresses.
2. Establish a connection with a remote system.
3. Copy itself to the remote system and cause the copy to be run.

- The network worm may also attempt to determine whether a system has previously been infected before copying itself to the system.
- As with viruses, network worms are difficult to counter.

2 VIRUS COUNTERMEASURES

2.1 Antivirus Approaches

- The ideal solution to the threat of viruses is prevention: Do not allow a virus to get into the system in the first place, or block the ability of a virus to modify any files containing executable code or macros. This goal is, in general, impossible to achieve, although prevention can reduce the number of successful viral attacks. The next best approach is to be able to do the following:
    - **Detection:** Once the infection has occurred, determine that it has occurred and locate the virus.
    - **Identification**: Once detection has been achieved, identify the specific virus that has infected a program.

- **Removal**: Once the specific virus has been identified, remove all traces of the virus from the infected program and restore it to its original state. Remove the virus from all infected systems so that the disease cannot spread further.

➢ If detection succeeds but either identification or removal is not possible, then the alternative is to discard the infected program and reload a clean backup version.

➢ There are four generations of antivirus software:

1. First generation: simple scanners
2. Second generation: heuristic scanners
3. Third generation: activity traps
4. Fourth generation: full-featured protection

1. **A first-generation scanner**
   - Requires a virus signature to identify a virus... Such signature-specific scanners are limited to the detection of known viruses.
   - Another type of first-generation scanner maintains a record of the length of programs and looks for changes in length.
2. **A second-generation scanner**
   - Does not rely on a specific signature. Rather, the scanner uses heuristic rules to search for probable virus infection. One class of such scanners looks for fragments of code that are often associated with viruses.
   - Another second-generation approach is integrity checking. A checksum can be appended to each program.
   - If a virus infects the program without changing the checksum, then an integrity check will catch the change. To counter a virus that is sophisticated enough to change the checksum when it infects a program, an encrypted hash function can be used.
   - The encryption key is stored separately from the program so that the virus cannot generate a new hash code and encrypt that.
   - By using a hash function rather than a simpler checksum, the virus is prevented from adjusting the program to produce the same hash code as before.
3. **Third-generation programs**
   - Are memory-resident programs that identify a virus by its actions rather than its structure in an infected program. Such programs have the advantage that it is not necessary to develop signatures and heuristics for a wide array of viruses.

- Rather, it is necessary only to identify the small set of actions that indicate an infection is being attempted and then to intervene.

4. **Fourth-generation products**
   - Are packages consisting of a variety of antivirus techniques used in conjunction. These include scanning and activity trap components.
   - In addition, such a package includes access control capability, which limits the ability of viruses to penetrate a system and then limits the ability of a virus to update files in order to pass on the infection.
   - The arms race continues. With fourth-generation packages, a more comprehensive defense strategy is employed, broadening the scope of defense to more general-purpose computer security measures.

2.2 Advanced Antivirus Techniques

More sophisticated antivirus approaches and products continue to appear. In this subsection, we highlight two of the most important.

2.2.1 Generic Decryption

Generic decryption (GD) technology enables the antivirus program to easily detect even the most complex polymorphic viruses, while maintaining fast scanning speeds. In order to detect such a structure, executable files are run through a GD scanner, which contains the following elements:

- **CPU emulator:** A software-based virtual computer. Instructions in an executable file are interpreted by the emulator rather than executed on the underlying processor. The emulator includes software versions of all registers and other processor hardware, so that the underlying processor is unaffected by programs interpreted on the emulator.
- **Virus signature scanner:** A module that scans the target code looking for known virus signatures.
- **Emulation control module:** Controls the execution of the target code.

2.2.2 Digital Immune System

The digital immune system is a comprehensive approach to virus protection developed by IBM]. The motivation for this development has been the rising threat of Internet-based virus propagation. Two major trends in Internet technology have had an increasing impact on the rate of virus propagation in recent years:

1. **Integrated mail systems:** Systems such as Lotus Notes and Microsoft Outlook make it very simple to send anything to anyone and to work with objects that are received.
2. **Mobile-program systems:** Capabilities such as Java and ActiveX allow programs to move on their own from one system to another.

Figure 5:-

illustrates the typical steps in digital immune system operation:

1. A monitoring program on each PC uses a variety of heuristics based on system behavior, suspicious changes to programs, or family signature to infer that a virus may be present. The monitoring program forwards a copy of any program thought to be infected to an administrative machine within the organization.
2. The administrative machine encrypts the sample and sends it to a central virus analysis machine.

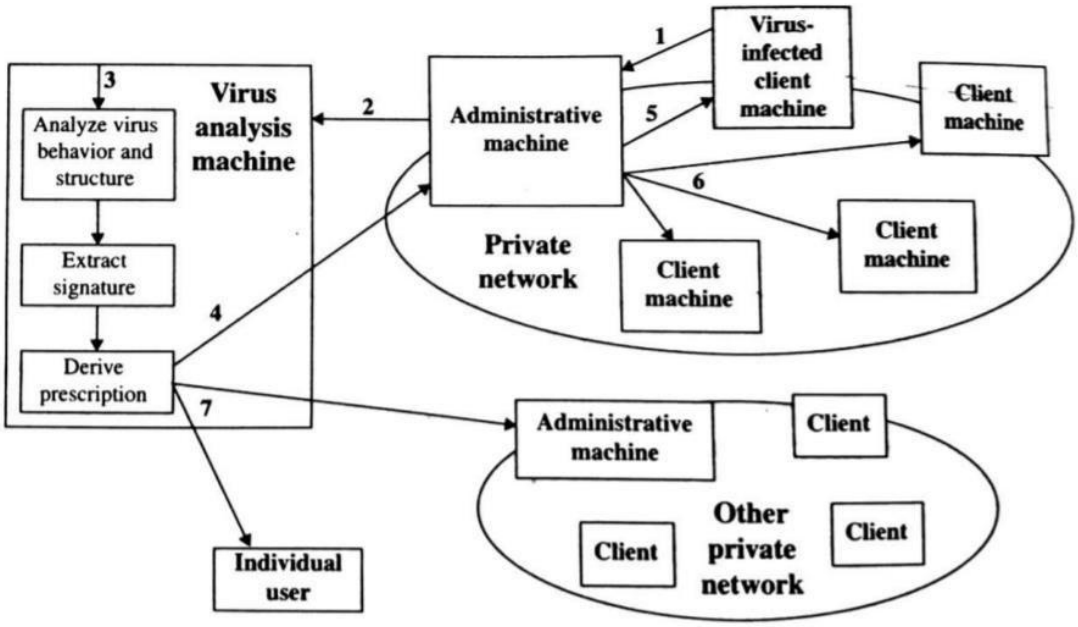

Figure 5 Digital Immune System

3. This machine creates an environment in which the infected program can be safely run for analysis. Techniques used for this purpose include emulation, or the creation of a protected environment within which the suspect program can be executed and monitored. The virus analysis machine then produces a prescription for identifying and removing the virus.
4. The resulting prescription is sent back to the administrative machine.
5. The administrative machine forwards the prescription to the infected client.
6. The prescription is also forwarded to other clients in the organization.
7. Subscribers around the world receive regular antivirus updates that protect them from the new virus.

The success of the digital immune system depends on the ability of the virus analysis machine to detect new and innovative virus strains. By constantly analyzing and monitoring the viruses found in the wild, it should be possible to continually update the digital immune software to keep up with the threat.

2.3 Behavior-Blocking Software

Unlike heuristics or fingerprint-based scanners, behavior-blocking software integrates with the operating system of a host computer and monitors program behavior in real-time for malicious actions. Monitored behaviors can include the following:

- Attempts to open, view, delete, and/or modify files;
- Attempts to format disk drives and other unrecoverable disk operations;
- Modifications to the logic of executable files or macros;
- Modification of critical system settings, such as start-up settings;
- Scripting of e-mail and instant messaging clients to send executable content; and
- Initiation of network communications.

If the behavior blocker detects that a program is initiating would-be malicious behaviors as it runs, it can block these behaviors in real-time and/or terminate the offending software. This gives it a fundamental advantage over such established antivirus detection techniques as fingerprinting or heuristics.

2.3 Behavior-Blocking Software

Unlike heuristics or fingerprint-based scanners, behavior-blocking software integrates with the operating system of a host computer and monitors program behavior in real-time for malicious actions. Monitored behaviors can include the following:

- Attempts to open, view, delete, and/or modify files;
- Attempts to format disk drives and other unrecoverable disk operations;
- Modifications to the logic of executable files or macros;
- Modification of critical system settings, such as start-up settings;
- Scripting of e-mail and instant messaging clients to send executable content; and
- Initiation of network communications.

If the behavior blocker detects that a program is initiating would-be malicious behaviors as it runs, it can block these behaviors in real-time and/or terminate the offending software. This gives it a fundamental advantage over such established antivirus detection techniques as fingerprinting or heuristics.